Assignment — C# Fundamentals

Community Library Inventory System

Generative AI (GenAI) Use

GenAI Level 1 — Limited AI Use ⚠️

This assignment is classified as **Level 1 — Limited AI Use**. Please follow the **Level 1 GenAI requirements** described in the *Generative AI Use in Assessments* policy posted at the top of the Assignments section in Moodle.

Scenario

You have been hired as a junior .NET developer by a **community library**. The library currently keeps a small inventory of its books manually and would like a simple C# console application that can organize and display information about its collection.

Your task is to develop a **Community Library Inventory System** as a .NET Console Application using C#.

The purpose of this assignment is to practice the core C# concepts covered in class, including classes, namespaces, objects, predefined types, value and reference types, nullable reference types, generic collections, boxing/unboxing, string interpolation, and pattern matching. These concepts correspond directly to the introductory C# material covered in this module.

Part 1 — Create the Project

Using Visual Studio, create a new:

Console App (.NET)

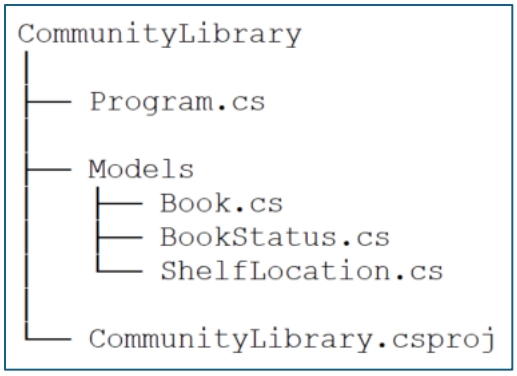
Use a modern version of .NET supported by your Visual Studio installation.

Name the project:

`CommunityLibrary`

Organize the application using multiple .cs files. The course material emphasizes organizing important types into separate source files to keep a project maintainable.

A possible organization is:



Part 2 — Create the `Book` Class

Create a `Book` class in a separate file and appropriate namespace.

Each book must contain at least the following properties:

```
Id
Title
Author
Price
PublicationYear
ISBN
Status
```

Choose appropriate C# data types for each property.

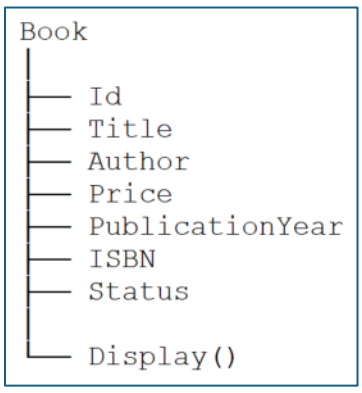
The `ISBN` should be allowed to contain `null` when the ISBN is unknown. Use an appropriate nullable reference type for this requirement.

Add a method named:

`Display()`

The method should display the information about the book using Console.WriteLine() and **string interpolation**.

Your class should therefore conceptually resemble:



Do not simply copy a completed implementation. Determine the appropriate types and implementation yourself.

Part 3 — Create an Enum

Create an `enum` named:

`BookStatus`

It should represent at least three possible states:

```
Available
Borrowed
Reserved
```

Assign an appropriate status to each book.

Enums are value types in C#, along with numeric types and structs.

Part 4 — Create a Struct

Create a struct named:

`ShelfLocation`

It should contain information identifying where a book is physically located, such as:

```
Aisle
Shelf
```

Create at least two `ShelfLocation` variables and demonstrate **value-type assignment behavior**.

For example:

```
Location 1
     ↓
copy
     ↓
Location 2
```

Modify the second variable after the assignment and demonstrate that changing it does not change the first variable.

Part 5 — Create Book Objects

In `Program.cs`, create at least **five different Book objects**.

Use realistic data.

For example, the objects might conceptually contain:

```
Book 1
Title: ...
Author: ...
Price: ...
Status: Available

Book 2
Title: ...
Author: ...
Price: ...
Status: Borrowed
```

At least one book must have:

```
ISBN = null
```

Use object initialization when creating the objects.

Part 6 — Store Books in a Generic Collection

Create a:

```
List<Book>
```

and place your five books inside the collection.

Use a loop to display every book by calling its `Display()` method.

> Generic collections provide compile-time type safety and normally avoid the casting and boxing issues associated with older object-based collections such as `ArrayList`.

Your program should produce output similar in structure to:

```
=== COMMUNITY LIBRARY ===

Book ID: 1
Title: ...
Author: ...
Price: ...
Status: Available

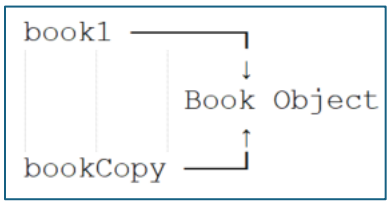
Book ID: 2
Title: ...
Author: ...
Price: ...
Status: Borrowed
```

Part 7 — Demonstrate Reference-Type Behavior

Because `Book` is a class, it is a **reference type**.

Select one of your book objects and assign it to another variable.

Conceptually:



Change the title or another property through the second reference.

Display the property through both variables and explain why both variables reflect the change.

The course material distinguishes this from value types: value-type assignment copies a value, while reference-type assignment copies the reference.

Part 8 — Handle a Nullable ISBN

Find a book whose ISBN is `null`.

Use a null check such as:

```
if (book.ISBN is not null)
{
    // ...
}
else
{
    // ...
}
```

Display an appropriate message in each case.

For example:

```
ISBN: 978-...
```

or:

```
ISBN: Not available
```

This requirement practices nullable reference types and null checking, which help detect potential null-reference problems earlier.

Part 9 — Demonstrate Boxing and Unboxing

Create an integer representing the number of books in the library:

```
int numberOfBooks = 5;
```

Box the value into an `object`.

Then unbox the value back into an `int`.

Display both values.

Your program should demonstrate the following process:

```
int
 ↓
Boxing
 ↓
object
 ↓
Unboxing
 ↓
int
```

Remember that boxing converts a value type to `object`, while unboxing explicitly extracts the value again.

Part 10 — Demonstrate the Unified Type System and Pattern Matching

Create several variables of type `object` containing different kinds of values, for example:

```
a number
a string
a decimal value
```

Use C# pattern matching to determine the type of at least one object.

For example:

```
if (value is string text)
{
    // ...
}
```

Display information about the detected value.

This demonstrates the relationship between C# types and **`System.Object`**, as well as basic type-pattern matching.

Part 11 — Calculate an Inventory Summary

Using the books stored in your collection, calculate and display:

```
Total number of books
Total value of all books
Average book price
```

Display the results using string interpolation and appropriate numeric formatting.

Example:

```
=== INVENTORY SUMMARY ===
```

```
Number of Books: 5
Total Inventory Value: $...
Average Book Price: $...
```

You may use loops and basic arithmetic. **LINQ is not required for this assignment.**

Part 12 — Comments and Code Quality

Add appropriate comments to important sections of your application.

At least one class or method should contain an XML documentation comment:

```
/// <summary>
/// ...
/// </summary>
```

Comments should explain the purpose or reasoning behind the code rather than simply repeating what the code already says, consistent with the course material.

Testing Requirements

Run the application and verify that all requirements work correctly.

Your final execution should demonstrate, at minimum:

- creation and display of five books;
- use of a `Book` class;
- use of `BookStatus`;
- use of `ShelfLocation`;
- generic `List<Book>`;
- nullable ISBN handling;
- value-type behavior;
- reference-type behavior;
- boxing and unboxing;
- pattern matching;
- inventory calculations;
- formatted console output.

The application should compile and run without errors.

Report and Screenshots

In addition to the Visual Studio project, submit a **report documenting your practical work**.

The report must contain a **comprehensive set of screenshots** demonstrating that you personally completed and tested the practical requirements.

Include screenshots showing the project structure, `Book` class, enum, struct, creation of book objects, `List<Book>`, nullable reference handling, value/reference type demonstrations, boxing/unboxing, pattern matching, calculations, successful compilation, and final console output.

Screenshots must be **clear, readable, logically ordered, and accompanied by a short explanation or caption**.

Do not submit only screenshots of source code. Your screenshots should also demonstrate that the application was actually executed and tested.

Submission Requirements

Submit the following through **Moodle**:

1. Your complete Visual Studio C# project/source code.
2. A report that includes a comprehensive set of screenshots demonstrating the implementation, execution, and testing of the practical requirements.

Respect the due date specified on Moodle. Late submissions will be handled according to the course policies.

Mandatory Presentation

In addition to submitting your work on Moodle, you must **present and demonstrate the assignment to the professor**.

During the presentation, you may be asked to:

- run your application;
- explain parts of your code;
- explain the difference between value and reference types;
- explain boxing and unboxing;
- identify the purpose of a class, struct, enum, or nullable reference;
- make a small modification to your program and run it again.

An assignment submitted on Moodle but not presented to the professor will not be graded.

Failure to include the required GenAI Disclosure Statement will result in a grade of zero (0) for the assignment.

Rubric

Criterion	Weight
C# Project Structure, Classes, Objects, and Types	25%
Collections and Required Application Functionality	20%
Value/Reference Types, Nullable Types, Boxing/Unboxing, and Pattern Matching	25%
Testing, Execution, and Code Quality	15%
Report and Comprehensive Screenshots	15%
Total	**100%**